Natural Language Processing Lab

Assignment 1: Dataset Collection and Text Processing

Indian Institute of Technology Jammu
Department of Computer Science and Engineering
Course Code: CS-5-45(MO) Semester 2 – 2025

Student: Vraj Patel **Id:** 2022UCS0100 **Dataset:** vrajpat3ll/arxiv-cs-papers
GitHub: vrajpat3ll/natural-language-processing

Task 1 — Data Collection

Domain and Source

The corpus was collected from the **arXiv preprint server** via the official `arxiv` Python client library, targeting Computer Science abstracts across four thematic categories. The domain was chosen for two reasons: (i) the text is formal, dense, and domain-specific, making it a challenging testbed for NLP methods; (ii) arXiv provides a clean, structured API with well-defined category labels.

Label Scheme

Four balanced classes were constructed by mapping arXiv subject codes:

Label	arXiv codes (examples)	AG News analogue
NLP/AI	cs.CL, cs.AI, cs.LG	World
Vision	cs.CV, eess.IV	Sports
Systems/Infra	cs.DC, cs.OS, cs.NI	Business
Algorithms	cs.DS, cs.CC, cs.CR	Sci/Tech

Table 1. Global dataset statistics.

Property	Value
Number of Documents	20,000
Number of Sentences	152,434
Average Sentence Length	27.88 tokens
Total Tokens	4,249,323
Source	via `arxiv` Python client

Table 2. Per-label statistics (5,000 documents per class).

Label	Docs	Sent.	Avg Len	Tokens
Algorithms	5,000	38,574	31.41	1,211,438
NLP/AI	5,000	37,207	26.80	997,258
Systems/Infra	5,000	37,577	27.03	1,015,828
Vision	5,000	39,076	26.23	1,024,799
Total	**20,000**	**152,434**	**27.88**	**4,249,323**

Observation. The Algorithms class yields noticeably longer sentences (31.41 tokens) compared to the other three classes (≈26–27 tokens). This is consistent with the denser, equation-heavy prose typical of theoretical CS writing. The dataset is perfectly class-balanced (25% per label), eliminating the need for class-weighting in downstream classifiers.

Task 2 — Text Cleaning

The following preprocessing pipeline was applied using NLTK and standard Python:

1. **Lowercasing:** All text converted to lowercase.
2. **URL and HTML removal:** Regex patterns strip `https?://\S+` and `<[^>]+>` tags.
3. **Special character removal:** Non-alphabetic characters (except spaces) removed.
4. **Whitespace normalisation:** Consecutive spaces collapsed to single space and stripped.
5. **Sentence segmentation:** NLTK `sent_tokenize` (Punkt model).
6. **Tokenisation:** NLTK `word_tokenize`; only alphabetic tokens retained.

After cleaning, the processed corpus contains **2,450,954 tokens** across **151,881 sentences**. The reduction from 4.25M raw tokens reflects removal of LaTeX math notation, citation strings (e.g. `et al.`, arXiv IDs), and special symbols common in scientific abstracts.

Example: Raw vs. Cleaned

Listing 1. Raw abstract excerpt (Algorithms class).

```
We study driver's optimal trajectory planning under uncertainty in the duration of a traffic light's green
    phase. We interpret this as an optimal control problem with an objective of minimizing the expected cost
    based on the fuel use, discomfort from rapid velocity changes, and time to destination. Treating this in
    the framework of dynamic programming, we show that the probability distribution on green phase durations
    gives rise to a sequence of Hamilton-Jacobi-Bellman PDEs, which are then solved numerically to obtain
    optimal acceleration/braking policy in feedback form. Our numerical examples illustrate the approach and
    highlight the role of conflicting goals and uncertainty in shaping drivers' behavior. [2201.04521].
```

Listing 2. After cleaning and tokenisation (stopwords retained).

```
['study', 'driver', 'optimal', 'trajectory', 'planning', 'uncertainty', 'duration', 'traffic', 'light', '
    green', 'phase', 'interpret', 'optimal', 'control', 'problem', 'objective', 'minimizing', 'expected', '
    cost', 'based', 'fuel', 'use', 'discomfort', 'rapid', 'velocity', 'changes', 'time', 'destination', '
    treating', 'framework', 'dynamic', 'programming', 'show', 'probability', 'distribution', 'green', 'phase
    ', 'durations', 'gives', 'rise', 'sequence', 'hamilton', 'jacobi', 'bellman', 'pdes', 'solved', '
    numerically', 'obtain', 'optimal', 'acceleration', 'braking', 'policy', 'feedback', 'form', 'numerical',
    'examples', 'illustrate', 'approach', 'highlight', 'role', 'conflicting', 'goals', 'uncertainty', '
    shaping', 'drivers', 'behavior']
```

Task 3 — Normalisation

Three normalisation strategies were applied and compared:

Table 3. Comparison of normalisation methods on a sample token sequence.

Method	Output (sample tokens)
Original (cleaned)	propose algorithm center problem metric spaces improving prior
Stopword removal	propose algorithm center problem metric spaces improving prior
Stemming (Porter)	propos algorithm center problem metric space improv prior
Lemmatisation (WordNet)	propose algorithm center problem metric space improving prior

Discussion.

- *Stopword removal* has limited impact on scientific abstracts since common stopwords (*the*, *is*) are already rare in technical writing; most tokens are content-bearing.
- *Stemming* (Porter) is aggressive—*propose* → *propos*, *improving* → *improv*—and can hurt readability and semantic coherence. Useful for high-recall IR tasks.
- *Lemmatisation* preserves valid dictionary forms (*spaces* → *space*) while being less destructive. Preferred for topic modelling (Lab 3) and embedding tasks (Labs 5–6).

Task 4 — Vocabulary Analysis

Table 4. Vocabulary statistics (post-cleaning, stopwords retained).

Property		Value
Total tokens (with repetition)		2,450,954
Vocabulary size (unique types)		49,971
Hapax legomena	16,703	(33.43% of vocabulary)
Average token frequency		49.05
Most frequent token		*models* (15,156)

Table 5. Top 20 most frequent tokens.

Rank	Token	Freq	Rank	Token	Freq
1	models	15,156	11	language	8,152
2	model	13,594	12	results	7,706
3	based	13,358	13	learning	7,696
4	data	12,223	14	across	7,579
5	time	11,678	15	large	7,391
6	performance	10,068	16	show	7,311
7	algorithm	9,433	17	using	7,180
8	problem	9,001	18	propose	7,140
9	framework	8,927	19	training	6,499
10	network	8,918	20	high	6,415

Frequency Histogram

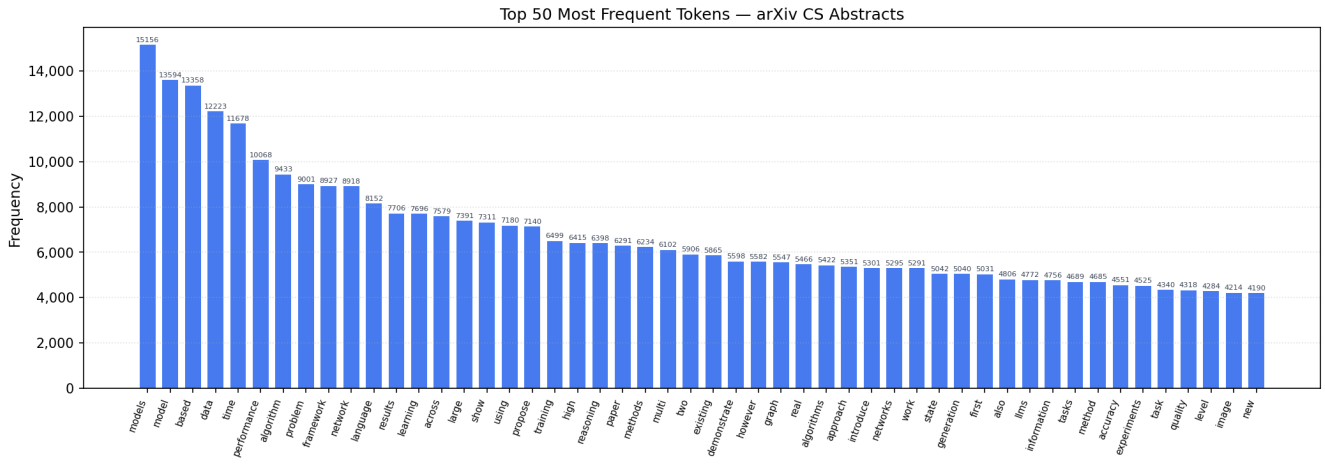


Figure 1. Top 50 most frequent tokens in the arXiv CS corpus. The steep drop-off from *models* (15,156) to *new* (4,190) across 50 tokens is characteristic of the Zipfian distribution. Domain-specific terms (*algorithm*, *framework*, *language*, *training*) dominate over generic function words, confirming that technical abstracts are information-dense.

Zipf's Law

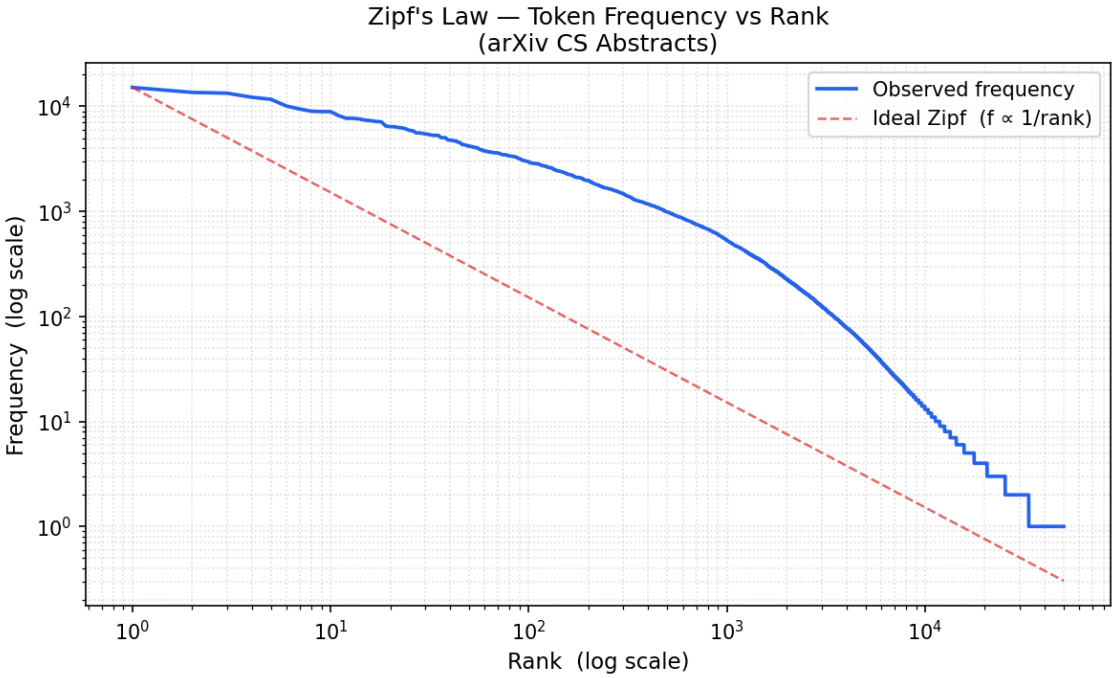


Figure 2. Log–log plot of token frequency vs. rank (blue) against the ideal Zipf curve $f \propto 1/r$ (dashed red). The corpus follows Zipf's Law in the mid-rank range (ranks 20–1,000) but deviates at both extremes.

Observations from Figures 1 and 2.

1. **High-rank deviation (top tokens):** The observed curve lies *above* the ideal Zipf line for ranks 1–20. This is common in domain-specific corpora where a handful of domain keywords (*model*, *algorithm*, *network*) are drastically overrepresented relative to general English.
2. **Mid-range fit:** Ranks 20–1,000 follow the power-law slope closely, confirming the corpus obeys Zipf's Law in the bulk.
3. **Low-rank tail deviation:** The sharp drop at ranks >10,000 reflects the high hapax fraction (33.4%). Words appearing only once form a discrete tail that cuts off more steeply than a continuous power law predicts.

Task 5 — N-gram Analysis

Table 6. N-gram coverage statistics.

N-gram	Unique count	Hapax %
Unigrams	49,971	33.4%
Bigrams	1,180,585	77.2%
Trigrams	1,898,026	94.6%

Table 7. Top 10 bigrams and trigrams by frequency.

Top Bigrams			Top Trigrams		
Rank	**Bigram**	**Freq**	**Rank**	**Trigram**	**Freq**
1	language models	4,278	1	large language models	2,973
2	state art	3,489	2	language models llms	1,963
3	large language	3,441	3	vision language models	591
4	real world	2,664	4	state art performance	536
5	models llms	1,968	5	achieves state art	533
6	extensive experiments	1,733	6	large language model	458
7	real time	1,677	7	extensive experiments demonstrate	436
8	large scale	1,599	8	experimental results demonstrate	384
9	fine tuning	1,495	9	outperforms state art	377
10	reinforcement learning	1,367	10	state art methods	377

Observations.

- The trigram *large language models* (2,973 occurrences) is the dominant phrase, reflecting the LLM wave that characterises post-2022 CS literature.
- *state art* is the stemmed/stopword-removed form of *state-of-the-art*; this pipeline artefact should be accounted for during feature engineering in Lab 3.
- Bigram hapax rate (77.2%) and trigram hapax rate (94.6%) confirm extreme data sparsity. Nearly all observed trigrams appear exactly once, making higher-order language modelling reliant on smoothing or backoff.
- Domain cluster terms (*reinforcement learning*, *fine tuning*, *vision language models*) naturally delineate the four categories, providing strong discriminative signal for the Naive Bayes classifiers in Lab 3.

Task 6 — Train / Validation / Test Split

A **stratified split** was performed using `sklearn.model_selection.train_test_split` with `RANDOM_SEED = 42` to ensure full reproducibility across all downstream labs. The splits were saved as CSV files and also published to HuggingFace Hub.

Table 8. Dataset split sizes.

Split	Documents	% of total
Train	16,000	80.0%
Validation	2,000	10.0%
Test	2,000	10.0%
Total	**20,000**	**100.0%**

Table 9. Class distribution across splits (stratified on `label`).

Class	Train	Val	Test	Train share
Algorithms	4,000	500	500	25.0%
NLP/AI	4,000	500	500	25.0%
Systems/Infra	4,000	500	500	25.0%
Vision	4,000	500	500	25.0%

Stratification ensures each split preserves the 25% per-class balance, preventing any partition from being enriched or depleted in a particular category. All future labs load pre-split data via a single `load_dataset("vrajpat3ll/arxiv-cs-papers"` call.

Appendix — Reproducibility

All experiments use `RANDOM_SEED = 42`. Plots are written to `plots/` and splits to `data/splits/`. The published HuggingFace dataset (`vrajpat3ll/arxiv-cs-papers`) contains all the samples.

Read the README.md file to find the commands you can run.